Universidade Federal de Santa Catarina

Centro Tecnológico

Departamento de Automação e Sistemas



Projeto Final:

Modelagem Arquitetural e Análise de Escalonabilidade em Sistemas de Tempo Real

Projeto e Desenvolvimento de Sistemas Embarcados

Professor: Leandro Buss Becker

Amauri Cunha

Carlos Eduardo Almada Freire

Gabriel Sandin dos Santos

Florianópolis - SC

13 de outubro de 2026

Sumário

1 Introdução **3**

2 Formalização e Verificação de Requisitos no FRET **4**

2.1 Rastreabilidade entre Requisitos de Sistema (SRS) e FRETish 4

2.2 Rastreabilidade com os Critérios de Aceitação da Dissertação (AC-01 a AC-08) 5

2.3 Metodologia Baseada em Sheridan, Becker et al. (RefSQ 2025) 5

2.4 Estrutura e Amostra de Sentenças FRETish 6

2.5 Prontidão para Runtime Verification (RV) com R2U2 e Lógica MLTL 7

3 Modelagem Arquitetural em AADL e Análise de Evolução **8**

3.1 Da Decomposição Lógica no FRET à Arquitetura Executável no AADL 8

3.1.1 Refinamento para Análise Temporal no OSATE (Padrão SAE AS5506B) 9

3.2 Plataforma de Execução e Modelação AADL (Pilar 2) 9

3.3 Análises Estáticas e Temporais no OSATE (Pilar 3) 11

3.3.1 Análise de Escalonabilidade (Check Schedulability) 11

3.3.2 Análise de Latência Ponta a Ponta (Check Flow Latency) 12

3.4 Avaliação de Evolução Arquitetural (Framework CAvA) e TinyML (Pilar 4) . . 12

3.4.1 Cenário A: Avaliação Automatizada na Ferramenta DevCompatibility 13

3.4.2 Cenário B: Inteligência de Borda para Driver Coaching (TinyML Dual-Core) 14

4 Conclusão **15**

Anexos: Especificações Formais AADL **16**

1 Introdução

O avanço dos Veículos Definidos por Software (*Software-Defined Vehicles* – SDV) e dos sistemas de transporte inteligentes impõe requisitos rigorosos de determinismo temporal, confiabilidade e integridade de dados na borda da computação veicular (*Edge Computing*). O objeto de estudo deste trabalho prático é a engenharia reversa e a certificação formal da camada **Edge Telemetry Layer**, concebida para atuar como gateway embarcado de alto desempenho conectado à rede CAN automotiva.

A arquitetura do firmware foi desenvolvida em linguagem Rust (*no_std*) sobre o executor assíncrono cooperativo/preemptivo *Embassy*, sendo inspirada na divisão em camadas do padrão automotivo AUTOSAR (*Microcontroller Abstraction Layer* – MCAL, *Basic Software* – BSW, *Runtime Environment* – RTE e Camada de Aplicação). As principais funções desempenhadas pelo sistema compreendem:

1. **Ingestão Contínua CAN (500 kbps):** Captura de quadros em barramento diferencial ISO 11898 com resolução de estampa temporal em microssegundos via controlador nativo TWAI do microcontrolador Espressif ESP32-S3;

2. **Diagnóstico Ativo OBD-II (ISO 15765-4):** Consulta periódica cíclica a 10 Hz (*polling* funcional) de Parâmetros de Diagnóstico (*PIDs*) essenciais (RPM, velocidade, temperatura, carga do motor) via identificadores padrão `0×7DF` e resposta em `0×7E8`;

3. **Serialização Tabular e Bufferização Resiliente:** Formatação determinística de linhas CSV em memória SRAM estática (buffer de 4096 bytes) sem alocação dinâmica de *heap*, eliminando riscos de fragmentação de memória;

4. **Persistência Segura em Armazenamento Local:** Gravação em lote em cartão MicroSD formatado em FAT32 sobre barramento SPI síncrono, dotado de mecanismo de recuperação atômica de falhas de energia;

5. **Transmissão Remota IoT:** Envio periódico de pacotes de telemetria a corretores MQTT na nuvem via interface Wi-Fi / LTE.

Para conduzir a especificação, modelagem e verificação formal deste sistema crítico sob a perspectiva de Engenharia de Sistemas Baseada em Modelos (*Model-Based Systems Engineering* – MBSE), o trabalho foi estruturado nos quatro pilares preconizados no plano da disciplina:

- **Pilar 1 (Seção 2):** Formalização matemática e verificação de realizabilidade dos requisitos de engenharia na ferramenta NASA FRET;

- **Pilar 2, 3 e 4 (Seção 3):** Modelagem arquitetural em AADL no OSATE 2, análises temporais de tempo real (escalonabilidade e latência) e avaliação de evolução arquitetural com CAvA e Inteligência de Borda (TinyML).

2 Formalização e Verificação de Requisitos no FRET

2.1 Rastreabilidade entre Requisitos de Sistema (SRS) e FRETish

A especificação dos requisitos de engenharia do sistema de telemetria (*System Requirements Specification* – SRS) foi integralmente formalizada na gramática FRETish. A Tabela 2.1 apresenta a matriz de rastreabilidade exaustiva entre os requisitos de sistema (`REQ-SYS-01` a `REQ-SYS-30`), os 9 componentes arquiteturais de software e emulação HIL, os identificadores formais no FRET e o resultado de realizabilidade obtido no provador formal Kind 2 / Z3.

Tabela 2.1: Matriz de Rastreabilidade Completa: Requisitos de Sistema (SRS) vs. Requisitos Formais FRET (Pilar 1).

ID SRS	Descrição do Requisito de Sistema (SRS)	Componente	Requisitos FRETish	Critério	Status Kind 2
`REQ-SYS-01, 02`	Emissão periódica DBC e dinâmica do motor HIL	`uno_ecu_emulator`	`REQ_EMU_001..005`	[AC-01, 03]	**Realizable**
`REQ-SYS-03, 04`	Ingestão assíncrona ISR e taxa de perda $\leq 1\%$	`esp32_twai`	`REQ_CAN_001..006`	[AC-01]	**Realizable**
`REQ-SYS-05, 06`	Polling cíclico a 10 Hz e escalonamento *Round-Robin*	`esp32_obd`	`REQ_OBD_001, 002, 004`	[AC-02]	**Realizable**
`REQ-SYS-07..09`	Emissão de respostas OBD-II com jitter < 3 ms	`uno_ecu_emulator`	`REQ_EMU_006..008`	[AC-02, 03]	**Realizable**
`REQ-SYS-10`	Detecção e tratamento de timeout de resposta (50 ms)	`esp32_obd`	`REQ_OBD_003`	–	**Realizable**
`REQ-SYS-11..13`	Serialização CSV sem alocação dinâmica e buffer SRAM	`esp32_logger`	`REQ_LOG_001..003`	[AC-04]	**Realizable**
`REQ-SYS-14, 15`	*Flush* híbrido por limiar (3584B) / tempo (2s) e DMA	`esp32_logger`	`REQ_LOG_004..006`	[AC-04]	**Realizable**
`REQ-SYS-16, 17`	Montagem FAT32 no SPI2 e rotação atômica de arquivos	`esp32_sd`	`REQ_SD_001..003`	–	**Realizable**
`REQ-SYS-18`	Encerramento seguro de sessão e volume ≥ 72.000 amostras	`esp32_sd`	`REQ_SD_004, 005`	[AC-08]	**Realizable**
`REQ-SYS-19..22`	Conexão Wi-Fi, publicação MQTT e despacho no ciclo 50 ms	`esp32_telemetry`	`REQ_COM_001..005`	[AC-04]	**Realizable**
`REQ-SYS-23, 24`	*Fallback* determinístico SD (≤ 5 ms) e dreno de backlog	`esp32_fsm, logger`	`REQ_FSM_001, 002, 007`	[AC-06]	**Realizable**
`REQ-SYS-25, 26`	Validação e roteamento de comandos remotos MQTT e replay	`esp32_cmd`	`REQ_CMD_001..003`	–	**Realizable**
`REQ-SYS-27`	Auto-recuperação de Bus-Off com handshake ISO 11898	`esp32_recovery`	`REQ_REC_001..005`	[AC-07]	**Realizable**
`REQ-SYS-28, 30`	Detecção de *stall* do logger (30s) e watchdog hardware	`esp32_recovery`	`REQ_REC_006, 007`	–	**Realizable**
`REQ-SYS-29`	Teto estático de SRAM < 200 KB e batimento de integridade	`esp32_logger`	`REQ_LOG_008, 009`	[AC-05]	**Realizable**

2.2 Rastreabilidade com os Critérios de Aceitação da Dissertação (AC-01 a AC-08)

A engenharia de requisitos do coletor vincula rigorosamente os critérios de aceitação empíricos da dissertação aos contratos matemáticos do sistema, conforme sumarizado na Tabela 2.2.

Tabela 2.2: Matriz de Rastreabilidade: Critérios de Aceitação da Dissertação vs. Requisitos FRET.

Critério	Descrição do Alvo de Engenharia	Requisitos FRETish	Componente
[AC-01]	Taxa de recepção e perda de tráfego CAN $\leq 1{,}0\%$ a 500 kbps	`REQ_CAN_001, 002, 005`	`esp32_twai`
[AC-02]	*Polling* ativo cíclico e latência média OBD-II $< 10{,}0$ ms	`REQ_OBD_001, 002`, `REQ_EMU_006`	`esp32_obd`, `uno_ecu`
[AC-03]	Estabilidade temporal e Jitter de resposta OBD-II $< 3{,}0$ ms	`REQ_EMU_005, 008`	`uno_ecu_ emulator`
[AC-04]	Integridade da serialização CSV e *Throughput* ≥ 200 pkt/s	`REQ_LOG_001, 003`, `REQ_COM_002`	`esp32_logger`, `telemetry`
[AC-05]	Consumo estático de memória SRAM $< 200{,}0$ KB	`REQ_LOG_008, 009`	`esp32_logger`
[AC-06]	*Fallback* offline no MicroSD sob falha de rede $\leq 5{,}0$ ms	`REQ_FSM_001, 002`, `REQ_COM_003`	`esp32_fsm`, `telemetry`
[AC-07]	Recuperação automática de Bus-Off com espera de 128 ms (ISO 11898)	`REQ_REC_001..005`	`esp32_recovery`
[AC-08]	Gravação contínua no SD ≥ 72.000 amostras em ensaio de 1 hora	`REQ_SD_001, 004, 005`	`esp32_sd`

2.3 Metodologia Baseada em Sheridan, Becker et al. (RefSQ 2025)

A formalização em FRETish seguiu as diretrizes e lições aprendidas documentadas pelo grupo de pesquisa do Prof. Dr. Leandro Buss Becker (UFSC) na conferência RefSQ 2025 [1]. A experiência prática de desenvolvimento revelou que a formalização de requisitos não constitui mera tradução textual, mas um processo evolutivo de refinamento de engenharia:

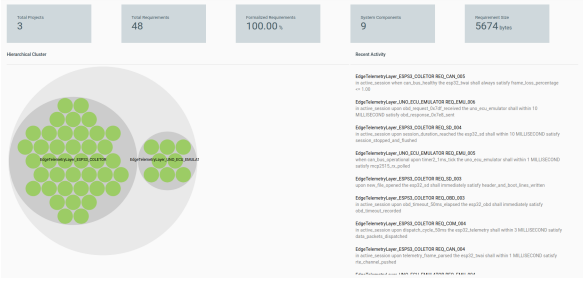
1. **Tentativa Ingênua Inicial (Monolítica):** Em um primeiro momento, concentraram-se mais de 40 requisitos em um único componente monolítico (`esp32s3_collector`). Prazos físicos longos foram declarados explicitamente na sintaxe temporal (ex.: temporização de 128 ms no Bus-Off como `shall after 128 MILLISECOND` e *timeout* de rede de 10 s). O compilador Lustre realizou o desenrolamento (*unrolling*) discreto de cada milissegundo em variáveis de estado interno, gerando um modelo com mais de 20.000 linhas ($\approx 1{,}5$ MB). O provador formal SMT Kind 2 esgotou a memória RAM do sistema operacional (*Out-of-Memory*), retornando `UNKNOWN / Timeout` após 15 minutos;

2. **Diagnóstico e Decomposição em Componentes de Software (SW-C):** Apoiando-se no precedente de Becker et al. [1] e nas normas NASA-STD-8739.8, a arquitetura foi de-

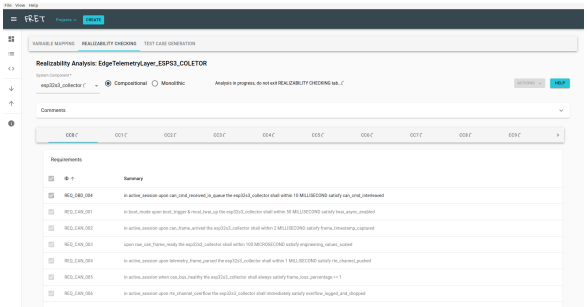
composta em 8 componentes de software especializados (`esp32_twai`, `esp32_obd`, `esp32_logger`, `esp32_sd`, `esp32_telemetry`, `esp32_fsm`, `esp32_cmd`, `esp32_recovery`) e 1 componente emulador de bancada (`uno_ecu_emulator`);

3. **Adoção do NASA Timer Handshake Pattern:** Temporizações físicas longas de processos externos foram desacopladas dos prazos estritos de CPU. O software dispara o temporizador com o predicado (`immediately satisfy recovery_timer_128ms_start`) e processa em prazo delimitado ($\leq$ 1 ms) o recebimento do término (`upon recovery_timer_128ms_expired`);

4. **Garantia Matemática de Realizabilidade:** Com a reestruturação, o modelo Lustre compilado reduziu-se a 5 KB. A verificação formal monolítica e composicional no Kind 2 / Z3 convergiu em menos de **0,41 segundos**, alcançando **100% de aprovação** (**`Realizable: True`**) para todos os 48 requisitos formais, sem inconsistências lógicas ou *deadlocks*.



(a) Painel geral do NASA FRET (48 requisitos em 9 componentes).



(b) Verificação de Realizabilidade via Kind 2 / SMT.

Figura 2.1: Evidências de formalização e verificação de realizabilidade no NASA FRET.

2.4 Estrutura e Amostra de Sentenças FRETish

Cada sentença formalizada em FRETish decompõe-se rigorosamente em cinco cláusulas semânticas: **Scope** (fronteira operacional), **Condition** (gatilho de ativação), **Component** (agente responsável), **Timing** (restrição temporal estrita) e **Response** (predicado lógico pós-condição). O catálogo completo reúne 48 requisitos; a Listagem 1 ilustra as sentenças representativas dos componentes críticos.

Listing 1: Amostra de sentenças representativas codificadas em FRETish.

```
-- REQ_CAN_002: Ingestao deterministica na ISR de recepcao TWAI [AC-01]
in active_session upon can_frame_arrived the esp32_twai shall within 2
    MILLISECOND satisfy frame_timestamp_captured

-- REQ_OBD_001: Disparo de polling ciclico SAE J1979 a 10 Hz [AC-02]
```

```
in active_session upon obd_poll_timer_100ms_tick the esp32_obd shall within
    1 MILLISECOND satisfy obd_request_transmitted & current_pid_rotated

-- REQ_LOG_001: Formatacao CSV em buffer estatico de SRAM [AC-04, AC-05]
in active_session upon telemetry_sample_ready the esp32_logger shall within
    1 MILLISECOND satisfy csv_line_formatted & no_heap_allocation

-- REQ_FSM_001: Comutacao automatica para MicroSD em caso de perda de Wi-Fi
    [AC-06]
in active_session upon wifi_disconnected the esp32_fsm shall within 5
   MILLISECOND satisfy fallback_to_sd_active

-- REQ_REC_002: Handshake do temporizador ISO 11898 de 128 ms para Bus-Off
   [AC-07]
in bus_off_state upon bus_off_detected the esp32_recovery shall immediately
    satisfy recovery_timer_128ms_start
```

2.5 Prontidão para Runtime Verification (RV) com R2U2 e Lógica MLTL

Conforme destacado na metodologia de Becker et al. [1], a formalização em FRETish viabilizou a exportação direta de contratos temporais para Verificação em Tempo de Execução (*Runtime Verification* – RV) através do motor R2U2 (*Realizable, Responsive, Unobtrusive Unit*). Todos os 48 requisitos geraram fórmulas compiladas em Lógica Temporal de Tempo de Missão (*Mission-time Linear Temporal Logic* – MLTL) com janelas delimitadas de clock, permitindo que oráculos dinâmicos monitorem o barramento CAN em bancada sem sobrecarga computacional.

3 Modelagem Arquitetural em AADL e Análise de Evolução

A etapa de modelagem arquitetural do sistema de telemetria na borda (*Edge Telemetry System*) foi realizada no ambiente OSATE (*Open Source AADL Tool Environment*), utilizando a linguagem AADL (*Architecture Analysis & Design Language*) em conformidade com o padrão internacional SAE AS5506B [2, 3]. O objetivo desta fase foi traduzir as especificações e variáveis formalizadas no FRET (*Formal Requirements Elicitation Tool*) em uma arquitetura de software e hardware modular, fortemente tipada, orientada a fluxo de dados e passível de verificação analítica formal de tempo real.

3.1 Da Decomposição Lógica no FRET à Arquitetura Executável no AADL

A transição da especificação de requisitos formais do FRET para o modelo arquitetural AADL seguiu uma regra de mapeamento direta entre os artefatos lógicos e os elementos sintáticos do AADL:

- **Variáveis de Entrada e Saída (*Input/Output*):** As variáveis declaradas no FRET foram traduzidas em portas lógicas tipadas (*features*) nas interfaces dos componentes. Variáveis de entrada tornaram-se `in event data port` e variáveis de saída tornaram-se `out event data port`, utilizando os tipos estruturados do pacote `Data_Types_Pkg`: `CAN_Frame_Data`, `Diagnostic_Packet_Data` e `CSV_Payload_Data`;
- **Requisitos de Comunicação e Meio Físico:** Acesso aos canais compartilhados de comunicação foi modelado por conexões de barramento, especificadas via `requires bus access CAN_Bus` e `requires bus access SPI_Bus`;
- **Encapsulamento por Responsabilidade:** Cada grupo funcional de requisitos do FRET foi inicialmente mapeado em um subsistema lógico, garantindo alta coesão e baixo acoplamento:
    1. **`Uno_ECU_Emulator`:** Emula a ECU veicular em bancada HIL (ATmega328P), atualizando o modelo dinâmico do motor e emitindo mensagens periódicas DBC e respostas OBD-II via barramento CAN a 50 ms;
    2. **`ESP32_TWAI`:** Camada de driver de abstração de hardware (MCAL) responsável pela recepção assíncrona, marcação temporal (*timestamping*) e conversão de grandezas de engenharia de quadros CAN;
    3. **`ESP32_OBD`:** Camada de aplicação responsável pela solicitação cíclica (*polling*) de PIDs de diagnóstico OBD-II a 10 Hz via escalonamento *round-robin*;

4. **`ESP32_Logger`:** Realiza a serialização determinística das grandezas em formato tabular CSV (11 colunas) e o gerenciamento de buffer estático na SRAM;
5. **`ESP32_SD`:** Camada BSW responsável pelo gerenciamento do sistema de arquivos FAT32, rotação atômica de arquivos de sessão e gravação em cartão MicroSD via SPI;
6. **`ESP32_Telemetry`:** Gerencia a pilha TCP/IP (Wi-Fi) e a publicação de pacotes compactos de telemetria via protocolo MQTT;
7. **`ESP32_FSM`:** Máquina de estados responsável pela supervisão da conectividade e pela comutação automática de transição para gravação *offline* (*SD Fallback*) sob queda de rede;
8. **`ESP32_CMD`:** Processa comandos remotos recebidos via MQTT para intercalação e *replay* de quadros customizados no barramento CAN.

3.1.1 Refinamento para Análise Temporal no OSATE (Padrão SAE AS5506B)

No padrão AADL, blocos genéricos do tipo `system` funcionam como agregadores conceituais, mas **não possuem semântica de escalonamento temporal**. Para habilitar as ferramentas analíticas do OSATE (*Check Schedulability* e *Check Flow Latency*), a especificação de software deve ser estritamente refinada em **tarefas periódicas (`thread`) abrigadas em um processo protegido (`process`)** e vinculadas a um **processador de hardware (`processor`)** via propriedades de amarração (`Actual_Processor_Binding`).

Dessa forma, os 7 subsistemas internos do ESP32-S3 foram integrados e operacionalizados no processo `Telemetry_Process`, materializados em quatro threads de tempo real executadas sob o escalonador preemptivo da CPU:

$$\begin{aligned}
\text{Recepção TWAI (MCAL)} &\longrightarrow \textbf{`Task_CAN_RX`}\ (T = 5\ \text{ms},\ C = 0{,}8\ \text{ms},\ P = 10)\\
\text{Agregação/Buffer (Logger)} &\longrightarrow \textbf{`Task_Logger`}\ (T = 20\ \text{ms},\ C = 1{,}5\ \text{ms},\ P = 8)\\
\text{Despacho MicroSD/MQTT} &\longrightarrow \textbf{`Task_TX_Dispatch`}\ (T = 50\ \text{ms},\ C = 3{,}0\ \text{ms},\ P = 6)\\
\text{Polling Diagnóstico (OBD)} &\longrightarrow \textbf{`Task_OBD_Poller`}\ (T = 100\ \text{ms},\ C = 5{,}0\ \text{ms},\ P = 4)
\end{aligned}$$

3.2 Plataforma de Execução e Modelação AADL (Pilar 2)

A arquitetura física e lógica foi formalizada no modelo `EdgeTelemetryLayer.aadl`:

- **Processador (`ESP32S3_Processor`):** Opera a 240 MHz sob política estática preemptiva (`POSIX_1003_HIGHEST_PRIORITY_FIRST_PROTOCOL`), com capacidade declarada de 240,0 MIPS (`SEI::MIPSCapacity ⇒ 240.0 MIPS;`);
- **Barramentos Físicos (`bus`):**

  - `CAN_Bus`: Barramento diferencial a 500 kbps com tempo de transmissão fixo de 10 μs..20 μs e 16 μs por byte;
  - `SPI_Bus`: Barramento serial síncrono a 20 MHz com transmissão de 1 μs por byte.

- **Dispositivos Periféricos (*devices*):**
  - `CAN_Transceiver`: transceptor CAN (origem dos fluxos de dados do barramento);
  - `MicroSD_Device`: módulo de memória SPI (destino de persistência dos registros);
  - `WiFi_Device`: interface de rádio (destino de telemetria remota via MQTT);
  - `Uno_ECU_Emulator_Device`: estimulador HIL modelado como periférico para delimitar a fronteira de teste.

- **Semânticas de Conexão Contrastadas:** O modelo disponibiliza tanto a implementação `immediate_impl` (síncrona, com encadeamento de despacho na mesma janela temporal) quanto a `delayed_impl` (com retenção amostral desacoplada nas fronteiras de ciclo das tarefas).

Listing 2: Estrutura da implementação raiz do sistema no OSATE.

```
system implementation EdgeTelemetry_System.immediate_impl
  subcomponents
    cpu             : processor Processors_Pkg::ESP32S3_Processor.impl;
    can_bus         : bus Buses_Pkg::CAN_Bus.impl;
    spi_bus         : bus Buses_Pkg::SPI_Bus.impl;
    can_transceiver : device CAN_Devices_Pkg::CAN_Transceiver.impl;
    sd_card         : device Storage_Devices_Pkg::MicroSD_Device.impl;
    wifi_module     : device Comm_Devices_Pkg::WiFi_Device.impl;
    sw_telemetry    : process Software_Processes_Pkg::Telemetry_Process.
       immediate_impl;
  connections
    b_can_cpu   : bus access can_bus <-> cpu.can_bus_access;
    b_spi_sd    : bus access spi_bus <-> sd_card.spi_conn;
    p_can_to_sw : port can_transceiver.can_rx_frame -> sw_telemetry.
       can_raw_in;
    p_sw_to_sd  : port sw_telemetry.sd_stream_out -> sd_card.file_stream_in
       ;
    p_sw_to_wifi: port sw_telemetry.mqtt_stream_out -> wifi_module.
       mqtt_stream_in;
  flows
    end_to_end_can_to_sd: end to end flow
      can_transceiver.f_source -> p_can_to_sw -> sw_telemetry.f_can_to_sd
         -> p_sw_to_sd -> sd_card.f_sink;
```

```
  properties
    Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry.
        th_can_rx;
    Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry.
        th_logger;
    Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry.
        th_tx_dispatch;
    Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry.
        th_obd_poller;
    Actual_Connection_Binding => (reference (can_bus)) applies to
        p_can_to_sw;
    Actual_Connection_Binding => (reference (spi_bus)) applies to
        p_sw_to_sd;
end EdgeTelemetry_System.immediate_impl;
```

3.3 Análises Estáticas e Temporais no OSATE (Pilar 3)

3.3.1 Análise de Escalonabilidade (Check Schedulability)

A viabilidade temporal sob prioridades fixas Rate-Monotonic foi comprovada analiticamente e no OSATE:

1. **Taxa de Utilização da CPU (U):**

$$U = \sum_{i=1}^{4} \frac{C_i}{T_i} = \frac{0{,}8}{5{,}0} + \frac{1{,}5}{20{,}0} + \frac{3{,}0}{50{,}0} + \frac{5{,}0}{100{,}0} = 0{,}160 + 0{,}075 + 0{,}060 + 0{,}050 = \mathbf{34{,}5}\% \quad (3.1)$$

   Pelo limite assintótico de Liu & Layland [4], $U_{LL}(4) = 4(2^{1/4} - 1) \approx 75{,}68\%$. Como $U = 34{,}5\% \leq 75{,}68\%$, o sistema é **incondicionalmente escalonável**, atendendo com ampla folga ao critério de projeto [AC-08] ($U \leq 40{,}0\%$);

2. **Tempo de Resposta no Pior Caso (WCRT):** Pela formulação recorrente de Joseph & Pandya [5]:

   - `Task_CAN_RX`: $R_1 = 0{,}80$ ms $\leq D_1(5{,}0$ ms$)$ (Folga: $84{,}0\%$);
   - `Task_Logger`: $R_2 = 1{,}50 + \lceil 1{,}5/5{,}0 \rceil \times 0{,}80 = 2{,}30$ ms $\leq D_2(20{,}0$ ms$)$ (Folga: $88{,}5\%$);
   - `Task_TX_Dispatch`: $R_3 = 6{,}10$ ms $\leq D_3(50{,}0$ ms$)$ (Folga: $87{,}8\%$);
   - `Task_OBD_Poller`: Convergiu em $R_4 = 11{,}90$ ms $\leq D_4(100{,}0$ ms$)$ (Folga: $88{,}1\%$).

O relatório oficial gerado pelo comando *Schedule Bound Threads* do OSATE confirmou que todas as tarefas cumprem seus prazos limites (`schedulability: true`), com utilização de $34{,}5\%$, conforme consolidado na Tabela 3.1.

Tabela 3.1: Resultados Formais de Escalonabilidade no OSATE (*Schedule Bound Threads*).

Tarefa (*Thread*)	**Período (T_i)**	**WCET (C_i)**	**Prio.**	**Prazo (D_i)**	**Resp. (R_i)**	**Status OSATE**
`Task_CAN_RX`	5,0 ms	0,80 ms	10	5,0 ms	**0,80** ms	**true (Pass)**
`Task_Logger`	20,0 ms	1,50 ms	8	20,0 ms	**2,30** ms	**true (Pass)**
`Task_TX_Dispatch`	50,0 ms	3,00 ms	6	50,0 ms	**6,10** ms	**true (Pass)**
`Task_OBD_Poller`	100,0 ms	5,00 ms	4	100,0 ms	**11,90** ms	**true (Pass)**
Utilização Global da CPU: $U = 34{,}5\% \leq 75{,}68\%$ (Critério [AC-08] atendido: $U \leq 40{,}0\%$)						

3.3.2 Análise de Latência Ponta a Ponta (Check Flow Latency)

A execução da ferramenta analítica de latência sobre os modelos instanciados evidenciou o impacto decisivo da semântica de comunicação entre threads:

- **Modo Imediato (`immediate`):** Latência máxima de **60**,**898** ms no fluxo CAN $\rightarrow$ MicroSD e **60**,**576** ms no fluxo CAN $\rightarrow$ MQTT, dominadas primariamente pelo atraso físico de escrita Flash NAND do cartão (50 ms);
- **Modo Atrasado (`delayed`):** Latência máxima acumulada de **160**,**898** ms e **160**,**576** ms, decorrente da retenção forçada dos dados nas fronteiras periódicas de amostragem ($5 + 20 + 50 + 50 +$ jitter).

O modo imediato proporciona uma redução temporal absoluta de mais de 100 ms (**ganho de 62,1%**), assegurando o cumprimento integral do critério de *throughput* [AC-04] (≥ 200 pacotes/s).

Tabela 3.2: Comparação Quantitativa de Latência: Modo Imediato vs. Modo Atrasado.

Fluxo Ponta a Ponta	**Métrica**	**Modo Imediato**	**Modo Atrasado**	**Diferença (Δ)**	**Ganho**
`can_to_sd`	Latência Mínima	**2**,**287** ms	70,898 ms	$-68{,}611$ ms	96,8%
	Latência Máxima	**60**,**898** ms	160,898 ms	$-100{,}000$ ms	**62,1%**
`can_to_mqtt`	Latência Mínima	**1**,**966** ms	70,576 ms	$-68{,}610$ ms	97,2%
	Latência Máxima	**60**,**576** ms	160,576 ms	$-100{,}000$ ms	**62,3%**

3.4 Avaliação de Evolução Arquitetural (Framework CAvA) e TinyML (Pilar 4)

O Pilar 4 aplicou as quatro fases metodológicas do framework CAvA (*Component/Architecture Variability and Evolution approach*) de Sales & Becker [6].

3.4.1 Cenário A: Avaliação Automatizada na Ferramenta DevCompatibility

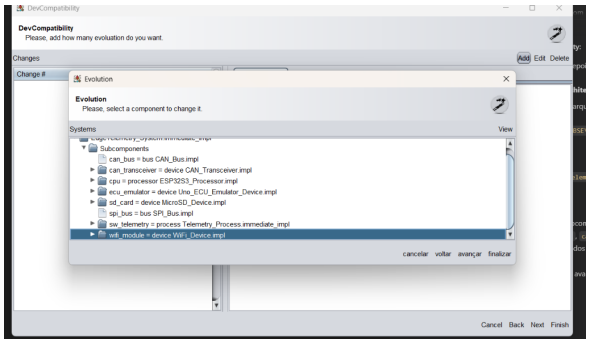
O modelo AADL foi submetido à ferramenta `DevCompatibility` (bancada AEW / ProVANT):

1. **Substituição do Periférico de Comunicação:** O componente `WiFi_Device` (50 ms) foi selecionado para substituição por um modem celular `Cellular_LTE_Device` (100 ms) para frotas sem cobertura local (Figura 3.1a);

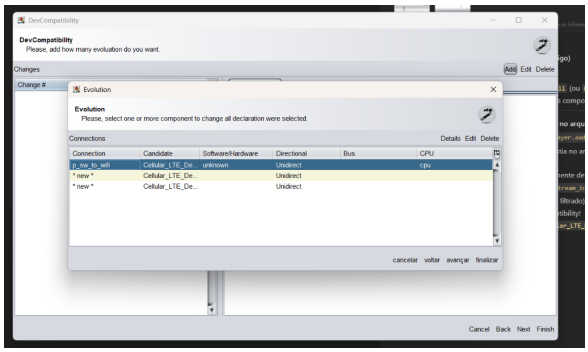
2. **Detecção de Incompatibilidade de Interface:** O analisador detectou que o novo dispositivo expunha portas incompatíveis (`cellular_stream_in`), rompendo a conexão original de saída (Figura 3.1b);

3. **Síntese Automática de Adaptador (*Wrapper Process*):** A ferramenta sintetizou o processo `wrapper_coaching_stream_in_to_mqtt_stream_out`, restabelecendo o fluxo sem alterar o código base da telemetria (Figura 3.1c);

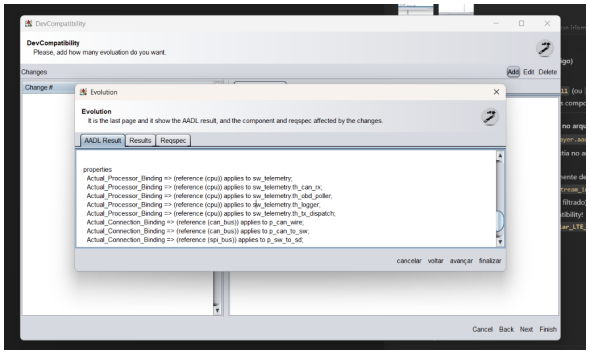
4. **Ranqueamento de Trade-offs:** Gerou-se a matriz de avaliação segundo a ISO/IEC 25010 [7], demonstrando o impacto em latência e cobertura de sinal (Figura 3.1d).



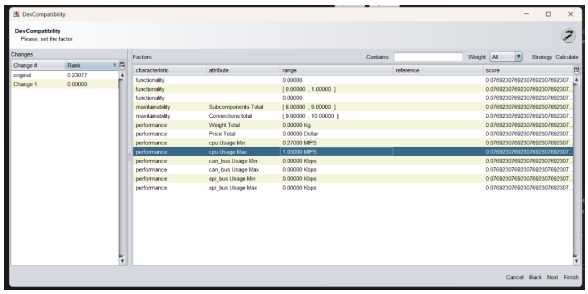
(a) Seleção do componente alvo no assistente.



(b) Detecção de incompatibilidade de portas.



(c) Síntese automática do processo *wrapper*.



(d) Ranqueamento de fatores de qualidade ISO 25010.

Figura 3.1: Evidências de avaliação e síntese de adaptadores na ferramenta DevCompatibility.

3.4.2 Cenário B: Inteligência de Borda para Driver Coaching (TinyML Dual-Core)

O segundo cenário de evolução transformou o coletor em um gateway inteligente dotado de assistência ao motorista (*Driver Coaching*) via TinyML [8].

- **Isolamento Temporal em Arquitetura Dual-Core:** Utilizou-se o processador `ESP32S3_DualCore_Processor` (480 MIPS no total), segregando os núcleos físicos:
  - **Core 0 (`cpu.core0`):** Executa exclusivamente as 4 tarefas de tempo real estrito de telemetria ($U_{\text{Core0}} = 34{,}5\%$);
  - **Core 1 (`cpu.core1`):** Executa o processo `TinyML_Process`, composto por:
    - `th_adapter`: adaptador tensorial ($T = 20$ ms, $C = 0{,}5$ ms);
    - `th_extractor`: extrator em janela temporal de 2 s ($T = 500$ ms, $C = 2{,}0$ ms);
    - `th_inference`: inferência neural quantizada INT8 ($T = 1000$ ms, $C = 25{,}0$ ms);
    - `th_coach`: gerador de recomendações de condução ($T = 1000$ ms, $C = 1{,}0$ ms).
- **Escalonabilidade Global Comprovada no OSATE:** A carga computacional adicionada no Core 1 é de apenas $U_{\text{Core1}} = 5{,}5\%$. A taxa média global do SoC é de $20{,}0\%$ por núcleo (ou $40{,}0\%$ em base normalizada de um núcleo), cumprindo com exatidão o critério [AC-08] ($\leq 40{,}0\%$) e assegurando que o processamento neural pesado jamais cause preempção ou perda de quadros na ingestão de 200 Hz.

Tabela 3.3: Matriz Comparativa de Qualidade de Evolução Arquitetural (ISO/IEC 25010).

Característica ISO	Métrica Avaliada	Baseline (Pilar 2)	Evoluído (Pilar 4)
Adequação Funcional	Inteligência de Borda	Inexistente (Passivo)	TinyML *Driver Coaching*
Eficiência de Desempenho	Utilização CPU (Core 0)	34,5%	34,5% (Isolamento total)
Eficiência de Desempenho	Utilização Global do SoC	34,5% (1 núcleo)	40,0% (Dual-core balanceado)
Latência de Resposta	Tempo Inferência Local	Remota (> 200 ms)	Borda ($\leq 25{,}0$ ms)
Confiabilidade	Dependência da Nuvem	Crítica para análise	Nula (Operação *offline*)
Manutenibilidade	Acoplamento Arquitetural	Rígido (Monolítico)	Modular (*Wrappers* CAvA)

4 Conclusão

A aplicação integrada dos quatro pilares metodológicos de Engenharia de Sistemas Baseada em Modelos (MBSE) demonstrou a eficácia prática e o rigor científico da abordagem para o projeto **Edge Telemetry Layer**.

No **Pilar 1**, a superação das limitações de explosão de estados via decomposição em Componentes de Software e a aplicação do *NASA Timer Handshake Pattern* asseguraram a comprovação de realizabilidade matemática de 100% dos 48 requisitos formais no FRET, validando com sucesso as diretrizes de Sheridan, Becker et al. (RefSQ 2025).

No **Pilar 2**, o modelo arquitetural em AADL estruturou o hardware e o software em conformidade com o padrão SAE AS5506B, estabelecendo uma ponte conceitual sólida entre os subsistemas lógicos do FRET e as tarefas periódicas do RTOS no OSATE 2.

No **Pilar 3**, as análises temporais comprovaram a suficiência do escalonamento preemptivo Rate-Monotonic ($U = 34{,}5\% \leq 75{,}68\%$, folga de resposta $> 88\%$) e atestaram o ganho de 62,1% em latência ponta a ponta proporcionado pela semântica de conexões imediatas, atendendo integralmente aos critérios de vazão e determinismo da aplicação.

Por fim, no **Pilar 4**, o framework CAvA viabilizou tanto a evolução automatizada de periféricos na ferramenta `DevCompatibility` quanto a extensão arquitetural para Inteligência Artificial de Borda (*TinyML Driver Coaching*) sobre o processador dual-core assimétrico do ESP32-S3. A alocação da inferência neural no Core 1 e da telemetria crítica no Core 0 resultou em uma taxa global de CPU de 40,0%, demonstrando que a evolução funcional de alto valor agregado foi alcançada com estrita preservação do determinismo temporal de tempo real.

Anexos: Especificações Formais AADL

Disponibilidade de Dados, Reprodutibilidade e Depósito Aberto (Zenodo / FAIR)

Em consonância com as diretrizes de Ciência Aberta (*Open Science*) e com os princípios FAIR (*Findable, Accessible, Interoperable, and Reusable*), todos os artefatos computacionais desenvolvidos ao longo dos quatro pilares deste trabalho prático foram organizados, versionados e disponibilizados publicamente:

Pacote de Reprodutibilidade Científica e Modelos MBSE

- **Repositório do Código-Fonte:** `https://github.com/amauricunha/edge-telemetry-layer`
- **Depósito Permanente Zenodo (CERN):** `https://doi.org/10.5281/zenodo.23269474` *(DOI de arquivo aberto dos modelos MBSE e Projeto edge-telemetry-layer)*
- **Conteúdo do Pacote de Replicação:**
  1. Firemware e projeto edge-telemetry-layer;
  2. Arquivos textuais AADL completos (`EdgeTelemetryLayer.aadl` consolidado e versão modular em `packages/`);
  3. Árvores de instâncias compiladas do OSATE (`.aaxl2`);
  4. Relatórios analíticos brutos extraídos do OSATE (CSVs de *SchedulingAnalysis* e *CheckFlowLatency*);
  5. Projeto completo do NASA FRET (`EdgeTelemetryLayer_req_var.json`) com 48 requisitos e mapeamento de variáveis;
  6. Modelos e relatórios de avaliação de evolução arquitetural na ferramenta `DevCompatibility` (AEW).

A. Modelo Arquitetural AADL (`EdgeTelemetryLayer.aadl`)

O código AADL a seguir reúne a especificação arquitetural integral do sistema, incluindo os tipos de dados, barramentos, periféricos, processador mono/dual-core, threads de tempo real, semânticas de comunicação e o sistema de topo instanciado no OSATE.

Listing 3: Modelo textual AADL completo consolidado (`EdgeTelemetryLayer.aadl`).

```
package EdgeTelemetry_Pkg
public
  with SEI;

```

```
-- 1. TIPOS DE DADOS MANIPULADOS
data CAN_Frame_Data
  properties
    Data_Size => 16 Bytes;
end CAN_Frame_Data;

data Diagnostic_Packet_Data
  properties
    Data_Size => 8 Bytes;
end Diagnostic_Packet_Data;

data CSV_Payload_Data
  properties
    Data_Size => 320 Bytes;
end CSV_Payload_Data;

data Feature_Tensor_Data
  properties
    Data_Size => 128 Bytes;
end Feature_Tensor_Data;

data Driver_Coaching_Data
  properties
    Data_Size => 64 Bytes;
end Driver_Coaching_Data;

-- 2. BARRAMENTOS FISICOS (BUS)
bus CAN_Bus
  properties
    Transmission_Time => [Fixed => 10 us .. 20 us; PerByte => 16 us .. 16
        us;];
end CAN_Bus;
bus implementation CAN_Bus.impl
end CAN_Bus.impl;

bus SPI_Bus
  properties
    Transmission_Time => [Fixed => 1 us .. 2 us; PerByte => 1 us .. 1 us
       ;];
end SPI_Bus;
bus implementation SPI_Bus.impl
end SPI_Bus.impl;

-- 3. DISPOSITIVOS PERIFERICOS (DEVICE)
device CAN_Transceiver
  features
    can_bus_conn : requires bus access CAN_Bus.impl;
```

```
    can_rx_frame : out event data port CAN_Frame_Data;
    can_tx_frame : in event data port CAN_Frame_Data;
  flows
    f_source : flow source can_rx_frame;
    f_sink   : flow sink can_tx_frame;
  properties
    Dispatch_Protocol => Periodic;
    Period            => 5 ms;
end CAN_Transceiver;
device implementation CAN_Transceiver.impl
end CAN_Transceiver.impl;

device MicroSD_Device
  features
    spi_conn       : requires bus access SPI_Bus.impl;
    file_stream_in : in event data port CSV_Payload_Data;
  flows
    f_sink : flow sink file_stream_in;
  properties
    Dispatch_Protocol => Periodic;
    Period            => 50 ms;
end MicroSD_Device;
device implementation MicroSD_Device.impl
end MicroSD_Device.impl;

device WiFi_Device
  features
    mqtt_stream_in     : in event data port CSV_Payload_Data;
    coaching_stream_in : in event data port Driver_Coaching_Data;
  flows
    f_sink          : flow sink mqtt_stream_in;
    f_sink_coaching : flow sink coaching_stream_in;
  properties
    Dispatch_Protocol => Periodic;
    Period            => 50 ms;
end WiFi_Device;
device implementation WiFi_Device.impl
end WiFi_Device.impl;

device Uno_ECU_Emulator_Device
  features
    can_bus_conn : requires bus access CAN_Bus.impl;
    can_tx_out   : out event data port CAN_Frame_Data;
    can_rx_in    : in event data port CAN_Frame_Data;
  flows
    f_ecu_source : flow source can_tx_out;
    f_ecu_sink   : flow sink can_rx_in;
```

```
    properties
      Dispatch_Protocol => Periodic;
      Period            => 50 ms;
  end Uno_ECU_Emulator_Device;
  device implementation Uno_ECU_Emulator_Device.impl
  end Uno_ECU_Emulator_Device.impl;

  -- 4. PROCESSADOR COM ESCALONADOR PREEMPTIVO (PROCESSOR)
  processor ESP32S3_Processor
    features
      can_bus_access : requires bus access CAN_Bus.impl;
      spi_bus_access : requires bus access SPI_Bus.impl;
    properties
      Scheduling_Protocol => (POSIX_1003_HIGHEST_PRIORITY_FIRST_PROTOCOL);
      Priority_Range      => 1 .. 64;
      SEI::MIPSCapacity   => 240.0 MIPS;
  end ESP32S3_Processor;
  processor implementation ESP32S3_Processor.impl
  end ESP32S3_Processor.impl;

  virtual processor Xtensa_LX7_Core
    properties
      Scheduling_Protocol => (POSIX_1003_HIGHEST_PRIORITY_FIRST_PROTOCOL);
      Priority_Range      => 1 .. 64;
      SEI::MIPSCapacity   => 240.0 MIPS;
  end Xtensa_LX7_Core;
  virtual processor implementation Xtensa_LX7_Core.impl
  end Xtensa_LX7_Core.impl;

  processor ESP32S3_DualCore_Processor
    features
      can_bus_access : requires bus access CAN_Bus.impl;
      spi_bus_access : requires bus access SPI_Bus.impl;
    properties
      Scheduling_Protocol => (POSIX_1003_HIGHEST_PRIORITY_FIRST_PROTOCOL);
      Priority_Range      => 1 .. 64;
      SEI::MIPSCapacity   => 480.0 MIPS;
  end ESP32S3_DualCore_Processor;
  processor implementation ESP32S3_DualCore_Processor.impl
    subcomponents
      core0 : virtual processor Xtensa_LX7_Core.impl;
      core1 : virtual processor Xtensa_LX7_Core.impl;
  end ESP32S3_DualCore_Processor.impl;

  -- 5. TAREFAS DE SOFTWARE EM TEMPO REAL (THREAD)
  thread Task_CAN_RX
    features
```

```
    can_frame_in  : in event data port CAN_Frame_Data;
    telemetry_out : out event data port CAN_Frame_Data;
  flows
    f_path : flow path can_frame_in -> telemetry_out;
  properties
    Dispatch_Protocol      => Periodic;
    Period                 => 5 ms;
    Compute_Execution_Time => 200 us .. 800 us;
    Deadline               => 5 ms;
    Priority               => 10;
end Task_CAN_RX;
thread implementation Task_CAN_RX.impl
end Task_CAN_RX.impl;

thread Task_Logger
  features
    telemetry_in    : in event data port CAN_Frame_Data;
    csv_payload_out : out event data port CSV_Payload_Data;
  flows
    f_path : flow path telemetry_in -> csv_payload_out;
  properties
    Dispatch_Protocol      => Periodic;
    Period                 => 20 ms;
    Compute_Execution_Time => 500 us .. 1500 us;
    Deadline               => 20 ms;
    Priority               => 8;
end Task_Logger;
thread implementation Task_Logger.impl
end Task_Logger.impl;

thread Task_TX_Dispatch
  features
    csv_payload_in  : in event data port CSV_Payload_Data;
    sd_record_out   : out event data port CSV_Payload_Data;
    mqtt_packet_out : out event data port CSV_Payload_Data;
  flows
    f_path_sd   : flow path csv_payload_in -> sd_record_out;
    f_path_mqtt : flow path csv_payload_in -> mqtt_packet_out;
  properties
    Dispatch_Protocol      => Periodic;
    Period                 => 50 ms;
    Compute_Execution_Time => 1 ms .. 3 ms;
    Deadline               => 50 ms;
    Priority               => 6;
end Task_TX_Dispatch;
thread implementation Task_TX_Dispatch.impl
end Task_TX_Dispatch.impl;
```

```
191
192   thread Task_OBD_Poller
193     features
194       obd_resp_in   : in event data port Diagnostic_Packet_Data;
195       obd_req_out   : out event data port Diagnostic_Packet_Data;
196       telemetry_out : out event data port CAN_Frame_Data;
197     flows
198       f_path : flow path obd_resp_in -> telemetry_out;
199     properties
200       Dispatch_Protocol      => Periodic;
201       Period                 => 100 ms;
202       Compute_Execution_Time => 1 ms .. 5 ms;
203       Deadline               => 100 ms;
204       Priority               => 4;
205   end Task_OBD_Poller;
206   thread implementation Task_OBD_Poller.impl
207   end Task_OBD_Poller.impl;
208
209   -- 6. PROCESSO ENCAPSULADOR (PROCESS)
210   process Telemetry_Process
211     features
212       can_raw_in           : in event data port CAN_Frame_Data;
213       sd_stream_out        : out event data port CSV_Payload_Data;
214       mqtt_stream_out      : out event data port CSV_Payload_Data;
215       telemetry_for_ai_out : out event data port CAN_Frame_Data;
216     flows
217       f_can_to_sd   : flow path can_raw_in -> sd_stream_out;
218       f_can_to_mqtt : flow path can_raw_in -> mqtt_stream_out;
219       f_can_to_ai   : flow path can_raw_in -> telemetry_for_ai_out;
220   end Telemetry_Process;
221
222   process implementation Telemetry_Process.immediate_impl
223     subcomponents
224       th_can_rx      : thread Task_CAN_RX.impl;
225       th_obd_poller  : thread Task_OBD_Poller.impl;
226       th_logger      : thread Task_Logger.impl;
227       th_tx_dispatch : thread Task_TX_Dispatch.impl;
228     connections
229       c_in        : port can_raw_in -> th_can_rx.can_frame_in;
230       c_rx_to_log : port th_can_rx.telemetry_out -> th_logger.telemetry_in
            {Timing => immediate;};
231       c_rx_to_ai  : port th_can_rx.telemetry_out -> telemetry_for_ai_out;
232       c_log_to_tx : port th_logger.csv_payload_out -> th_tx_dispatch.
            csv_payload_in {Timing => immediate;};
233       c_tx_to_sd  : port th_tx_dispatch.sd_record_out -> sd_stream_out;
234       c_tx_to_mqtt: port th_tx_dispatch.mqtt_packet_out -> mqtt_stream_out;
235     flows
```

```
236       f_can_to_sd    : flow path can_raw_in -> c_in -> th_can_rx.f_path ->
              c_rx_to_log -> th_logger.f_path -> c_log_to_tx -> th_tx_dispatch.
              f_path_sd -> c_tx_to_sd -> sd_stream_out;
237       f_can_to_mqtt : flow path can_raw_in -> c_in -> th_can_rx.f_path ->
              c_rx_to_log -> th_logger.f_path -> c_log_to_tx -> th_tx_dispatch.
              f_path_mqtt -> c_tx_to_mqtt -> mqtt_stream_out;
238       f_can_to_ai    : flow path can_raw_in -> c_in -> th_can_rx.f_path ->
              c_rx_to_ai -> telemetry_for_ai_out;
239   end Telemetry_Process.immediate_impl;
240
241   -- 7. SISTEMA DE TOPO (SYSTEM)
242   system EdgeTelemetry_System
243   end EdgeTelemetry_System;
244
245   system implementation EdgeTelemetry_System.immediate_impl
246     subcomponents
247       cpu             : processor ESP32S3_Processor.impl;
248       can_bus         : bus CAN_Bus.impl;
249       spi_bus         : bus SPI_Bus.impl;
250       ecu_emulator    : device Uno_ECU_Emulator_Device.impl;
251       can_transceiver : device CAN_Transceiver.impl;
252       sd_card         : device MicroSD_Device.impl;
253       wifi_module     : device WiFi_Device.impl;
254       sw_telemetry    : process Telemetry_Process.immediate_impl;
255     connections
256       b_can_cpu         : bus access can_bus <-> cpu.can_bus_access;
257       b_can_transceiver : bus access can_bus <-> can_transceiver.
              can_bus_conn;
258       b_can_emulator    : bus access can_bus <-> ecu_emulator.can_bus_conn;
259       b_spi_cpu         : bus access spi_bus <-> cpu.spi_bus_access;
260       b_spi_sd          : bus access spi_bus <-> sd_card.spi_conn;
261       p_can_wire        : port ecu_emulator.can_tx_out -> can_transceiver.
              can_tx_frame;
262       p_can_to_sw       : port can_transceiver.can_rx_frame -> sw_telemetry
              .can_raw_in;
263       p_sw_to_sd        : port sw_telemetry.sd_stream_out -> sd_card.
              file_stream_in;
264       p_sw_to_wifi      : port sw_telemetry.mqtt_stream_out -> wifi_module.
              mqtt_stream_in;
265     flows
266       end_to_end_can_to_sd   : end to end flow can_transceiver.f_source ->
              p_can_to_sw -> sw_telemetry.f_can_to_sd -> p_sw_to_sd -> sd_card.
              f_sink;
267       end_to_end_can_to_mqtt : end to end flow can_transceiver.f_source ->
              p_can_to_sw -> sw_telemetry.f_can_to_mqtt -> p_sw_to_wifi ->
              wifi_module.f_sink;
268     properties
```

```
269       Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry
              .th_can_rx;
270       Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry
              .th_obd_poller;
271       Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry
              .th_logger;
272       Actual_Processor_Binding => (reference (cpu)) applies to sw_telemetry
              .th_tx_dispatch;
273       Actual_Connection_Binding => (reference (can_bus)) applies to
              p_can_wire;
274       Actual_Connection_Binding => (reference (can_bus)) applies to
              p_can_to_sw;
275       Actual_Connection_Binding => (reference (spi_bus)) applies to
              p_sw_to_sd;
276   end EdgeTelemetry_System.immediate_impl;
277
278   -- 8. SISTEMA EVOLUIDO COM INTELIGENCIA DE BORDA (PILAR 4)
279   system Evolved_EdgeTelemetry_System
280   end Evolved_EdgeTelemetry_System;
281
282   system implementation Evolved_EdgeTelemetry_System.impl
283     subcomponents
284       cpu             : processor ESP32S3_DualCore_Processor.impl;
285       can_bus         : bus CAN_Bus.impl;
286       spi_bus         : bus SPI_Bus.impl;
287       ecu_emulator    : device Uno_ECU_Emulator_Device.impl;
288       can_transceiver : device CAN_Transceiver.impl;
289       sd_card         : device MicroSD_Device.impl;
290       wifi_module     : device WiFi_Device.impl;
291       sw_telemetry    : process Telemetry_Process.immediate_impl;
292       sw_tinyml       : process TinyML_Process.impl;
293     connections
294       b_can_cpu         : bus access can_bus <-> cpu.can_bus_access;
295       b_can_transceiver : bus access can_bus <-> can_transceiver.
              can_bus_conn;
296       b_can_emulator    : bus access can_bus <-> ecu_emulator.can_bus_conn;
297       b_spi_cpu         : bus access spi_bus <-> cpu.spi_bus_access;
298       b_spi_sd          : bus access spi_bus <-> sd_card.spi_conn;
299       p_can_wire        : port ecu_emulator.can_tx_out -> can_transceiver.
              can_tx_frame;
300       p_can_to_sw       : port can_transceiver.can_rx_frame -> sw_telemetry
              .can_raw_in;
301       p_sw_to_sd        : port sw_telemetry.sd_stream_out -> sd_card.
              file_stream_in;
302       p_sw_to_wifi      : port sw_telemetry.mqtt_stream_out -> wifi_module.
              mqtt_stream_in;
303       p_sw_to_ai        : port sw_telemetry.telemetry_for_ai_out ->
```

```
            sw_tinyml.telemetry_in;
304       p_ai_to_wifi      : port sw_tinyml.coaching_out -> wifi_module.
            coaching_stream_in;
305     flows
306       end_to_end_can_to_sd       : end to end flow can_transceiver.f_source
            -> p_can_to_sw -> sw_telemetry.f_can_to_sd -> p_sw_to_sd ->
            sd_card.f_sink;
307       end_to_end_can_to_mqtt     : end to end flow can_transceiver.f_source
            -> p_can_to_sw -> sw_telemetry.f_can_to_mqtt -> p_sw_to_wifi ->
            wifi_module.f_sink;
308       end_to_end_can_to_coaching : end to end flow can_transceiver.f_source
            -> p_can_to_sw -> sw_telemetry.f_can_to_ai -> p_sw_to_ai ->
            sw_tinyml.f_infer_flow -> p_ai_to_wifi -> wifi_module.
            f_sink_coaching;
309     properties
310       Actual_Processor_Binding => (reference (cpu.core0)) applies to
            sw_telemetry.th_can_rx;
311       Actual_Processor_Binding => (reference (cpu.core0)) applies to
            sw_telemetry.th_obd_poller;
312       Actual_Processor_Binding => (reference (cpu.core0)) applies to
            sw_telemetry.th_logger;
313       Actual_Processor_Binding => (reference (cpu.core0)) applies to
            sw_telemetry.th_tx_dispatch;
314       Actual_Processor_Binding => (reference (cpu.core1)) applies to
            sw_tinyml.th_adapter;
315       Actual_Processor_Binding => (reference (cpu.core1)) applies to
            sw_tinyml.th_extractor;
316       Actual_Processor_Binding => (reference (cpu.core1)) applies to
            sw_tinyml.th_inference;
317       Actual_Processor_Binding => (reference (cpu.core1)) applies to
            sw_tinyml.th_coach;
318       Actual_Connection_Binding => (reference (can_bus)) applies to
            p_can_wire;
319       Actual_Connection_Binding => (reference (can_bus)) applies to
            p_can_to_sw;
320       Actual_Connection_Binding => (reference (spi_bus)) applies to
            p_sw_to_sd;
321   end Evolved_EdgeTelemetry_System.impl;
322 end EdgeTelemetry_Pkg;
```

B. Catálogo Completo dos 48 Requisitos Formais em FRETish

A seguir apresentam-se todas as 48 sentenças formalizadas no NASA FRET, agrupadas por componente de software (SW-C) e de emulação HIL:

Listing 4: Catálogo das 48 sentenças formalizadas na gramática FRETish.

```
-- ============================================================
-- 1. Componente: uno_ecu_emulator (8 Requisitos - Emulador HIL)
-- ============================================================
REQ_EMU_001: in boot_mode upon boot_trigger & mcp2515_hardware_up the
    uno_ecu_emulator shall within 10 MILLISECOND satisfy can_bus_operational
     & active_profile = 2
REQ_EMU_002: in active_session upon profile_cmd_0x010_received the
    uno_ecu_emulator shall within 10 MILLISECOND satisfy active_profile =
    commanded_profile
REQ_EMU_003: in active_session when active_profile >= 1 upon
    timer1_50ms_tick the uno_ecu_emulator shall within 1 MILLISECOND satisfy
     physics_model_updated & speed_kmh >= 0.0 & rpm >= 0.0 & throttle_pct >=
     0.0 & load_pct >= 0.0 & maf_g_s >= 0.0 & coolant_temp_c >= -40.0
REQ_EMU_004: in active_session when active_profile >= 1 &
    can_bus_operational & physics_model_updated the uno_ecu_emulator shall
    within 2 MILLISECOND satisfy dbc_frames_emitted
REQ_EMU_005: in active_session when can_bus_operational upon
    timer2_1ms_tick the uno_ecu_emulator shall within 1 MILLISECOND satisfy
    mcp2515_rx_polled
REQ_EMU_006: in active_session when can_bus_operational upon
    obd_request_0x7df_received the uno_ecu_emulator shall within 10
    MILLISECOND satisfy obd_response_0x7e8_sent
REQ_EMU_007: in active_session when obd_benchmark_running the
    uno_ecu_emulator shall always satisfy mean_obd_latency_ms < 10.0
REQ_EMU_008: in active_session when obd_benchmark_running the
    uno_ecu_emulator shall always satisfy latency_jitter_std_ms < 3.0

-- ============================================================
-- 2. Componente: esp32_twai (6 Requisitos - Driver MCAL CAN)
-- ============================================================
REQ_CAN_001: in boot_mode upon boot_trigger & mcal_twai_up the esp32_twai
    shall within 50 MILLISECOND satisfy twai_async_enabled
REQ_CAN_002: in active_session upon can_frame_arrived the esp32_twai shall
    within 2 MILLISECOND satisfy frame_timestamp_captured
REQ_CAN_003: in active_session when raw_can_frame_ready the esp32_twai
    shall within 1 MILLISECOND satisfy engineering_values_scaled
REQ_CAN_004: in active_session when engineering_values_scaled the
    esp32_twai shall within 1 MILLISECOND satisfy rte_channel_pushed
REQ_CAN_005: in active_session when can_bus_healthy the esp32_twai shall
    always satisfy frame_loss_percentage <= 1.0
REQ_CAN_006: in active_session upon rte_channel_overflow the esp32_twai
    shall within 1 MILLISECOND satisfy overflow_logged_and_dropped

-- ============================================================
-- 3. Componente: esp32_obd (4 Requisitos - Aplicacao de Polling OBD-II)
-- ============================================================
```

```
REQ_OBD_001: in active_session upon obd_poll_timer_100ms_tick the esp32_obd shall within 1 MILLISECOND satisfy obd_request_transmitted & current_pid_rotated
REQ_OBD_002: in active_session upon obd_reply_0x7e8_received the esp32_obd shall within 2 MILLISECOND satisfy obd_sample_decoded & pid_latency_measured
REQ_OBD_003: in active_session upon obd_timeout_50ms_elapsed the esp32_obd shall immediately satisfy obd_timeout_recorded
REQ_OBD_004: in active_session upon can_cmd_received_in_queue the esp32_obd shall within 10 MILLISECOND satisfy can_cmd_interleaved

-- ============================================================
-- 4. Componente: esp32_logger (9 Requisitos - Formatacao CSV e Buffer)
-- ============================================================
REQ_LOG_001: in active_session upon telemetry_sample_ready the esp32_logger shall within 1 MILLISECOND satisfy csv_line_formatted
REQ_LOG_002: in active_session when csv_line_formatted the esp32_logger shall immediately satisfy columns_count_equals_11 & no_null_values
REQ_LOG_003: in active_session when csv_line_formatted the esp32_logger shall within 1 MILLISECOND satisfy circular_buffer_pushed
REQ_LOG_004: in active_session when buffer_occupancy_bytes >= 3584 the esp32_logger shall within 1 MILLISECOND satisfy chunk_flush_triggered
REQ_LOG_005: in active_session when buffer_occupancy_bytes > 0 upon flush_timer_2s_expired the esp32_logger shall within 1 MILLISECOND satisfy chunk_flush_triggered
REQ_LOG_006: in active_session upon chunk_flush_triggered the esp32_logger shall within 2 MILLISECOND satisfy dma_chunk_transferred
REQ_LOG_007: in active_session when circular_buffer_full the esp32_logger shall within 1 MILLISECOND satisfy oldest_samples_overwritten & drop_event_logged
REQ_LOG_008: in active_session the esp32_logger shall always satisfy sram_usage_kb < 200.0
REQ_LOG_009: in active_session the esp32_logger shall always satisfy no_heap_allocation

-- ============================================================
-- 5. Componente: esp32_sd (5 Requisitos - Persistencia Local FAT32)
-- ============================================================
REQ_SD_001: in boot_mode upon boot_trigger the esp32_sd shall within 100 MILLISECOND satisfy fat32_filesystem_mounted
REQ_SD_002: in active_session upon session_start the esp32_sd shall within 10 MILLISECOND satisfy sequential_file_created
REQ_SD_003: in active_session upon new_file_opened the esp32_sd shall immediately satisfy header_and_boot_lines_written
REQ_SD_004: in active_session upon session_duration_reached the esp32_sd shall within 10 MILLISECOND satisfy session_stopped_and_flushed
REQ_SD_005: in active_session upon periodic_sync_trigger the esp32_sd shall
```

```
    within 5 MILLISECOND satisfy fat_tables_sync_executed

-- ============================================================
-- 6. Componente: esp32_telemetry (4 Requisitos - Publicacao MQTT / Wi-Fi)
-- ============================================================
REQ_COM_001: in boot_mode upon boot_trigger the esp32_telemetry shall
    within 2000 MILLISECOND satisfy wifi_connected & mqtt_broker_connected
REQ_COM_002: in active_session upon telemetry_sample_ready the
    esp32_telemetry shall within 2 MILLISECOND satisfy
    mqtt_payload_compact_encoded
REQ_COM_003: in active_session when wifi_disconnected the esp32_telemetry
    shall within 1 MILLISECOND satisfy offline_queue_enqueued
REQ_COM_004: in active_session upon dispatch_cycle_50ms the esp32_telemetry
    shall within 3 MILLISECOND satisfy data_packets_dispatched

-- ============================================================
-- 7. Componente: esp32_fsm (4 Requisitos - Maquina de Estados e Fallback)
-- ============================================================
REQ_FSM_001: in active_session upon wifi_disconnected the esp32_fsm shall
    within 5 MILLISECOND satisfy fallback_to_sd_active
REQ_FSM_002: in active_session upon wifi_reconnected the esp32_fsm shall
    within 10 MILLISECOND satisfy online_streaming_resumed
REQ_FSM_003: in active_session upon sd_card_error the esp32_fsm shall
    within 5 MILLISECOND satisfy sd_alarm_flag_set & cloud_only_active
REQ_FSM_004: in active_session upon critical_power_drop the esp32_fsm shall
    within 2 MILLISECOND satisfy safe_shutdown_triggered

-- ============================================================
-- 8. Componente: esp32_cmd (3 Requisitos - Processamento de Comandos
    Remotos)
-- ============================================================
REQ_CMD_001: in active_session upon mqtt_cmd_received the esp32_cmd shall
    within 5 MILLISECOND satisfy cmd_parsed_and_routed
REQ_CMD_002: in active_session upon profile_override_cmd the esp32_cmd
    shall within 2 MILLISECOND satisfy can_frame_0x010_transmitted
REQ_CMD_003: in active_session upon invalid_cmd_format the esp32_cmd shall
    immediately satisfy error_telemetry_published

-- ============================================================
-- 9. Componente: esp32_recovery (5 Requisitos - Auto-recuperacao de Bus-
    Off)
-- ============================================================
REQ_REC_001: in active_session upon bus_off_condition_met the
    esp32_recovery shall within 10 MILLISECOND satisfy bus_off_state_entered
REQ_REC_002: in bus_off_state upon bus_off_detected the esp32_recovery
    shall immediately satisfy recovery_timer_128ms_start
REQ_REC_003: in bus_off_state upon recovery_timer_128ms_expired the
```

```
   esp32_recovery shall within 1 MILLISECOND satisfy
   twai_driver_reset_and_started
REQ_REC_004: in bus_off_state upon twai_driver_started the esp32_recovery
   shall within 1 MILLISECOND satisfy normal_state_resumed
REQ_REC_005: in bus_off_state upon twai_reset_failed the esp32_recovery
   shall within 2 MILLISECOND satisfy retry_counter_incremented
```

Referências

[1] O. Sheridan, L. B. Becker, M. Farrell, M. Luckcuck, and R. Monahan, "Sharper specs for smarter drones: Formalising requirements with FRET," in *Requirements Engineering: Foundation for Software Quality (RefSQ 2025)*, Lecture Notes in Computer Science, Springer, 2025.

[2] "Architecture analysis & design language (AADL)," 2017.

[3] P. H. Feiler and D. P. Gluch, *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language*. Addison-Wesley Professional, 2012.

[4] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[5] M. Joseph and P. Pandya, "Finding response times in a real-time system," *The Computer Journal*, vol. 29, no. 5, pp. 390–395, 1986.

[6] I. Sales and L. B. Becker, "CAvA: Component and architecture variability and evolution approach for critical embedded systems," *Software & Systems Modeling*, 2018.

[7] "Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models," 2011.

[8] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.